Data Mining HW1

生資所 311351009 劉承霈

Q: How do you select features for your model input, and what preprocessing did you perform?

- Feature selection:

  At first, I applied the Pearson correlation coefficient to calculate the correlation between each feature and the label, and set a threshold greater than 0.5 to select strongly correlated features. However, the outcomes weren't satisfactory. Therefore, I directly chose PM2.5 as the only feature, and eventually got a better result.

- Data preprocessing:

  I. Handling Invalid Values:

  First, I created a list named `'invalid_values'` to store potential invalid values like #, *, x, A, and all these invalid values within the dataset were replaced with 'pd.NA', representing pandas `'NA/NaN'` value.

  II. Ensuring Numeric Data

  I converted all data in columns named '0' to '23' to a numeric format. Any non-numeric entries discovered during this process were coerced into NaN.

  III. Missing Value Imputation

  To address missing data, I implemented a function named `'fill_missing_values'` to handle missing data differently based on the column's content:

  - If the entire column was missing data, fill in these missing values by searching for values from previous periods.
  - Otherwise, first use the forward fill method `(ffill)`, which propagates the last known non-null value forward until another non-null value is encountered. If ffill leaves any values unfilled (which would happen if the missing values are at the beginning of the column), the backward fill method `(bfill)` is applied, which fills the values backward from the following known value.

IV. Collecting PM2.5 data

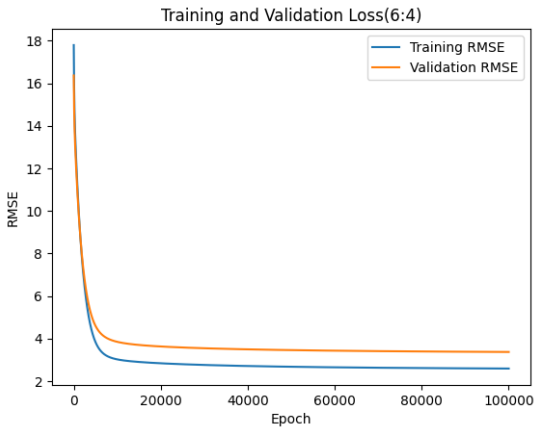
To systematically store PM2.5 data for each month, I initialized a dictionary named `'month_data_pm25'`. For each of the 12 months, a data matrix `'data_matrix_pm25'` was created with a single row representing PM2.5 values arranged across the hours of 20 days, resulting in 480 columns (20 days * 24 hours). This structure focuses solely on PM2.5 data, reflecting hourly measurements for each day.

To make the training set have the same format as the test set(train: 24 time points, test: 9 time points), I employed a sliding window, whose size is 9 hours, to go through these PM2.5 data, and picked the last day as the label.

V. Normalization

To improve the convergence during training and generalized model performance, I decided to normalize the feature data (`X_train`) used for forecasting PM2.5 levels. I first computed the mean (`X_mean`) and standard deviation (`X_std`) for each feature across the dataset, and then applied z-score normalization. This really helps in later result improvement.

Q: Compare the impact of different amounts of training data on the PM2.5 prediction accuracy. Visualize the results and explain them.

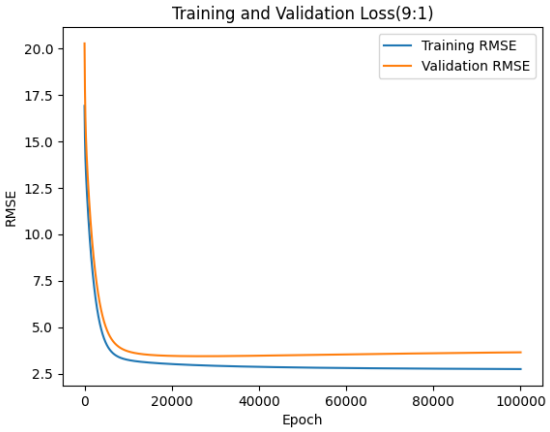



```
Training RMSE =

2.591485359534192

Validation RMSE =

3.370472343391008
```




```
Training RMSE =
2.7408928292300114,
Validation RMSE =
3.643561125226398
```

The two charts compare the RMSE for models with different training-validation splits. The first has a 6:4 split, and the second has a 9:1 split.

In the 6:4 split chart, we observe a close convergence between the training and validation RMSE. The model benefits from having a larger proportion of data for validation, providing a robust evaluation of model performance.

However, in the 9:1 split chart, the validation RMSE is considerably higher than in the 6:4 split. This suggests that the model, when trained with more data, might not generalize as well to new, unseen data due to the smaller validation set, indicating potential overfitting to the training data and less generalizability.

Q: Discuss the impact of regularization on PM2.5 prediction accuracy.

I used the L2 regularization in the gradient calculation during the training process **`(gradients = 2/X_train_b.shape[0] * X_train_b.T.dot(errors) + reg_lambda * self.weights)`**. By including this penalty, the model's complexity is reduced, which can prevent overfitting to the training data and help improve the model's generalization to new data.

Originally, I didn’t add regularization and my Leaderboard score could only pass a simple baseline. After applying regularization along with other hyperparameter adjustments, my Leaderboard score finally surpassed a strong baseline.